

Training Piscine JavaScript for backend – 1

Day01 – Strings & Arrays

Summary: Today, you will discover string and array manipulation: slicing, splitting, searching, template literals, and iterating over collections.

Version: 1.0

Contents

1	General rules	2
2	Specific instructions of the day	3
3	Exercise 00	4
4	Exercise 01	5
5	Exercise 02	6
6	Exercise 03	7
7	Exercise 04	8
8	Submission and peer-evaluation	9

Chapter 1

General rules

- You have to render your modules from a computer in the cluster, either using a virtual machine or the computer directly if the tools are available.
- Your functions should not crash unexpectedly. If this happens, your project will be considered non-functional and will receive a 0 during the evaluation.
- We encourage you to write test programs for your project, even though this work **won't have to be submitted and won't be graded**.
- Submit your work to your assigned Git repository. Only the work in the Git repository will be graded.
- You must use **Node.js (v18+)**.
- Your code must follow standard **ESLint** rules (2-space indentation, no global leakage).
- There is no global variable.

Chapter 2

Specific instructions of the day

- No code in the global scope outside of function definitions.
- Each file must contain **only** the requested function, plus its `module.exports`.
- Any exception not caught will invalidate the exercise.
- You can use any built-in function unless explicitly prohibited.
- All your functions must be documented using JSDoc comments where requested.

Chapter 3

Exercise 00

Exercise 00: Slice and split
Turn-in directory: ex00/
Files to turn in: ft_slice_split.js
Allowed functions: String.prototype.slice(), String.prototype.split(), console.log()

Write a function named `ft_slice_split` that takes a sentence as parameter, extracts the first word using `.slice()`, then splits the whole sentence into an array of words using `.split()`.

Function prototype:

```
/**
 * @param {string} sentence
 * @returns {void}
 */
function ft_slice_split(sentence) {
    // your code here
}
```

```
ft_slice_split("Node handles many requests");
First word: Node
Words: [ 'Node', 'handles', 'many', 'requests' ]
```

Reminder: `.slice(start, end)` extracts part of a string without modifying the original one. `.split(separator)` turns a string into an array, cut at every occurrence of separator.

Chapter 4

Exercise 01

Exercise 01: Replace and search
Turn-in directory: ex01/
Files to turn in: ft_replace_search.js
Allowed functions: String.prototype.replace(), String.prototype.includes(), template literals, console.log()

Write a function named `ft_replace_search` that takes a `name` and a `message` as parameters. It must:

- build a greeting using a **template literal** (``Hello ${name}``)
- replace the word "error" in `message` with "issue" using `.replace()`
- check whether `message` `.includes()` the word "urgent", and log a boolean

Function prototype:

```
/**
 * @param {string} name
 * @param {string} message
 * @returns {void}
 */
function ft_replace_search(name, message) {
  // your code here
}
```

```
ft_replace_search("Sara", "urgent error in the server");
Hello Sara
urgent issue in the server
Contains "urgent": true
```

Chapter 5

Exercise 02

Exercise 02: Array basics
Turn-in directory: ex02/
Files to turn in: ft_array_basics.js
Allowed functions: .length, array indexing [], console.log()

Write a function named `ft_array_basics` that takes an array as parameter and displays:

- the number of elements, using `.length`
- the first element, using index `[0]`
- the last element, using index `[array.length - 1]`

Function prototype:

```
/**
 * @param {Array} items
 * @returns {void}
 */
function ft_array_basics(items) {
    // your code here
}
```

```
ft_array_basics(["GET", "POST", "PUT", "DELETE"]);
Length: 4
First: GET
Last: DELETE
```

Chapter 6

Exercise 03

Exercise 03: Stack and queue
Turn-in directory: ex03/
Files to turn in: ft_stack_queue.js
Allowed functions: Array.prototype.push(), .pop(), .shift(), .unshift(), console.log()

Write a function named `ft_stack_queue` that takes an array as parameter and, on a **copy** of it, demonstrates the 4 core array mutation methods:

- `.push()` an element at the end
- `.pop()` the last element off
- `.unshift()` an element at the beginning
- `.shift()` the first element off

Log the array's content after each operation.

Function prototype:

```
/**
 * @param {Array} items
 * @returns {void}
 */
function ft_stack_queue(items) {
  // your code here
}
```

Warning: `.push()`, `.pop()`, `.shift()` and `.unshift()` all **mutate** the original array. If you need to keep the original untouched, work on a copy (e.g. with the spread operator `[...items]`).

Chapter 7

Exercise 04

Exercise 04: Loop it
Turn-in directory: ex04/
Files to turn in: ft_loop_it.js
Allowed functions: for, for...of, .length, array indexing [], console.log()

Write a function named `ft_loop_it` that takes an array of strings as parameter and prints every element twice: once using a classic `for` loop with indexing, once using a `for...of` loop.

Function prototype:

```
/**
 * @param {Array<string>} items
 * @returns {void}
 */
function ft_loop_it(items) {
  // your code here
}
```

```
ft_loop_it(["route", "middleware", "controller"]);
- for loop -
0: route
1: middleware
2: controller
- for...of loop -
route
middleware
controller
```

Reminder: a classic `for` loop gives you the **index** (useful with `items[i]`), while `for...of` gives you the **value** directly, without needing `.length` or indexing.

Chapter 8

Submission and peer-evaluation

Turn in your assignment in your Git repository as usual. Only the work inside your repository will be evaluated during the defense.

The evaluation process will happen on the computer of the evaluated group.